Cloud Industries External Device Guide

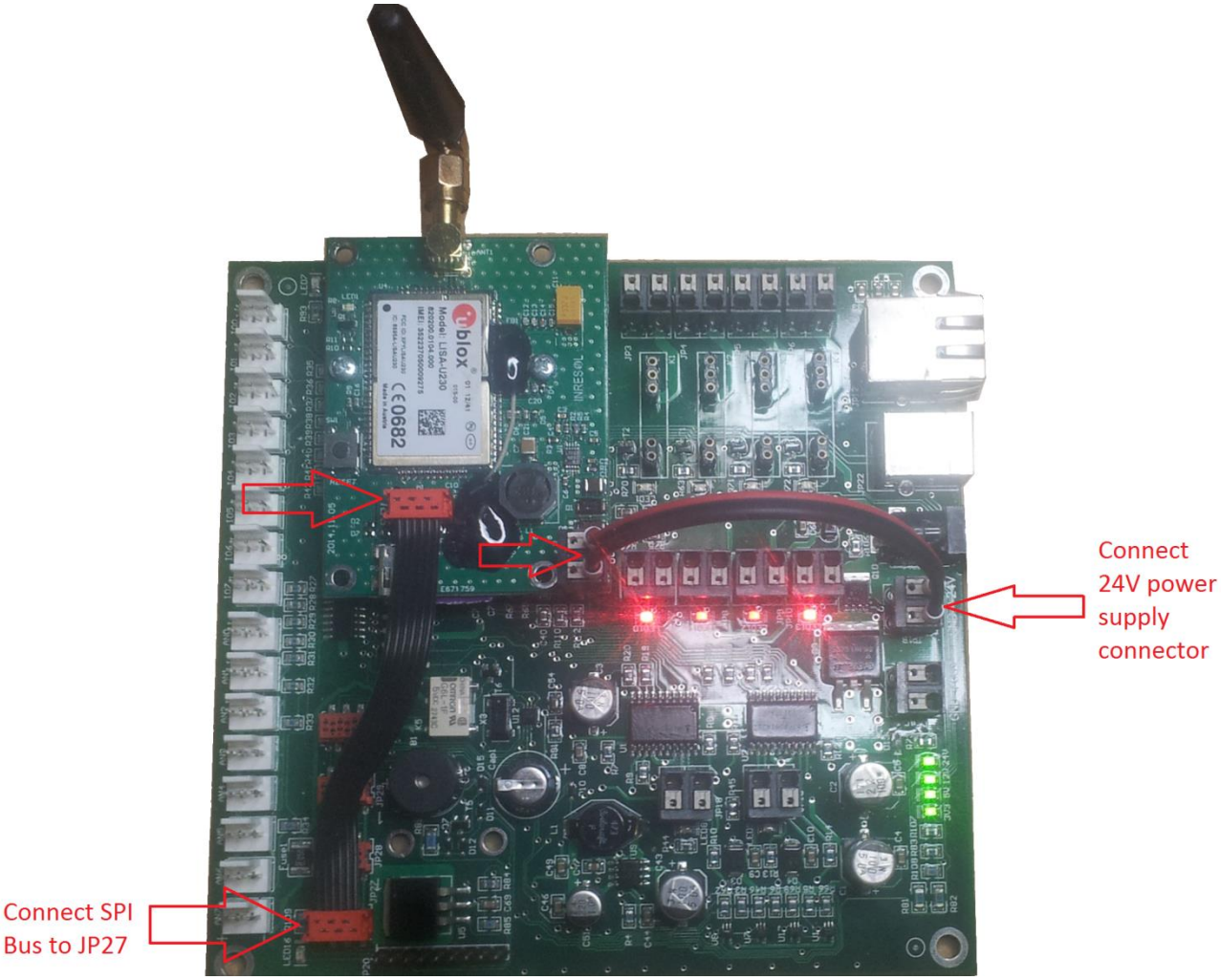
Contents

1. Hardware programming 2
   1.1. GSM Board connection 2
   1.2. Demo project startup 3
   1.3. Source code library implementation 3
   1.4. Cloud Industries connection library implementation notes 4
      1.4.1 Things to keep in mind 4
      1.4.2 Production 4
2. End-user login 6
   2.1 Generating DUID and PIN code pair 6
   2.2 User registration 6
3. Cloud Industries API 9
   3.1. Prerequisites 10
      3.1.1. API Key 10
      3.1.2. Unique device ID (DUID) 11
   3.2. API Documentation and testing environment. 11
   3.3. API Error logging 12

1. Hardware programming

1.1. GSM Board connection

For the demo project to run by default please connect GSM Prototype board as shown in Pict 1.1.



Pict 1.1. GSM Board connected to Melacs board.

All required wires are included with the prototype board. Please attach GSM antenna to the board and insert SIM card with PIN code disabled. Power up the board and connect it to your development computer.

1.2. Demo project startup

There is a demo project preconfigured to test your GSM board. To get the demo project running follow these steps:

- Extract the project files to a directory of your choice.
- Include project files to your MPLAB IDE.
- Ensure you have placed SIM card with PIN disabled into the GSM prototype board and made proper connections as in picture 1.1.
- Open main.c file and take a look at code line number 43, you should enter the your GSM provider APN string. If you are unaware of GSM provider's APN, please contact GSM service provider.
- Build and program the software to the Melacs board.
- It will take about 60 seconds for the GSM board to startup.
- If everything went fine LED should start to blink on the GSM board indicating that it has succesfully connected to the GSM service.
- Open your internet browser and go to https://inresol.cloudindustries.eu
- You should see a new device in the device list.
- Select „**Measurement list**" from the menu (Ref. Picture 1.3) you should see some of the measurements (for the demo we added only few of the measurements) changing – these are „randomly" generated values from the Melacs board.

The demo software may be triggering the Melacs board buzzer. Buzzer is triggered if a request to Cloud Industries server has failed. If it is sounding the buzzer constantly you should make sure that the SIM card has the possibility to connect to the internet over GSM network.

You should have a working system right now. If you have any trouble with the demo please contact <rokas@elseta.com>.

1.3.Source code library implementation

Cloud connection libraries should be included in your project. Please refer to the demo project for sample codes snippets. Cloud industries connection libraries cover SPI interface, AT command protocol for communication with U-Blox Lisa U-200 GSM module and data transfer using HTTPS protocol to Cloud Industries API.

1.4.Cloud Industries connection library implementation notes

1.4.1 Things to keep in mind

We have noticed a few issues while working with GSM that would be good to keep in mind while developing applications:

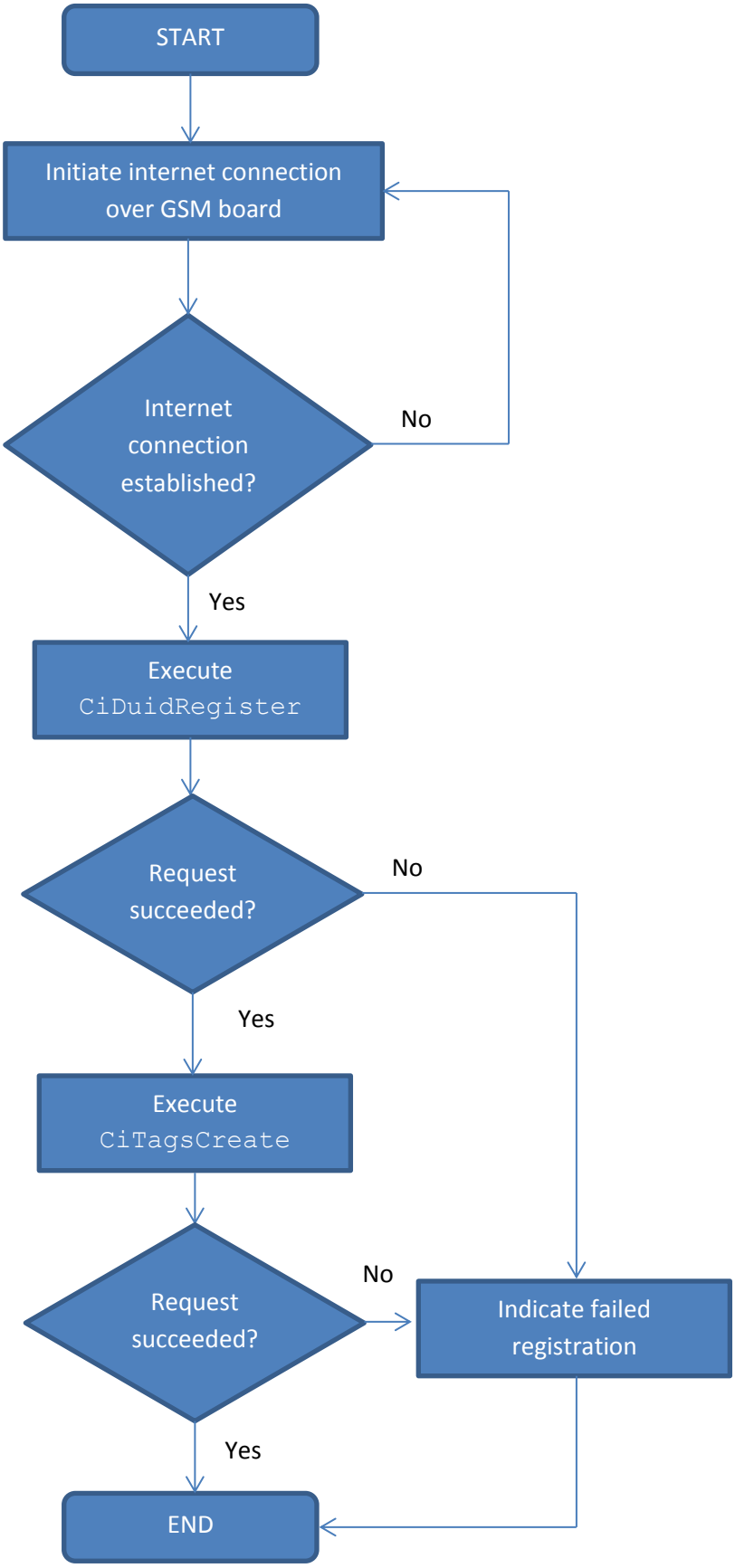
- Modem fails to boot up properly and may stop responding if AT commands are sent too soon after startup. Adding a long delay (60 seconds) before sending commands since power-on seems to fix this problem.
- There is no way to reset the modem with hardware if it stops responding. This is needed if it fails to boot or stops working after a long uptime.
- Sometimes AT command result is not received. In this case error is returned by LisaGetLine() after a 30 second timeout. We did not find the reason of this yet. It might be caused by modem event notifications or firmware/modem problems.
- API response text is not parsed or processed in any way. Command result is only determined by HTTP response code, but API documentation and API log can help to track any issues.
- Some functions use malloc() so make sure you have enough heap memory if firmware starts to act unpredictably.
- Measurement list update function is not yet implemented (check with rokas@elseta.com to implement this functionality when it will be required).

1.4.2 Production

When producing generators each device should send registration requests to our system. Library implements two functions which is designed to help you with the registration:

- `CiDuidRegister` – Device registration request, it registers the device on our system. Unique ID should be passed to this function.
- `CiTagsCreate` – Tag creation function. A tag is a measurement descriptor – tag represents measurement configuration (e.g. Real Power) and tags are created using templates. A string passed to this function is an alias for your device measurement and alarm template. For now you should always use *„api/inresol/genious-v0“* string. If measurement template is going to change we will prepare another version of the template with a new name.

These functions should be called only once per device, when the device boots up for the first time. If called later they should return failure. If both function succeed you should see a new device in the cloud industries front-end (https://inresol.cloudindustries.eu). Please refer to Pict. 1.2 for a flow diagram how the registration process should be implemented.



Pict. 1.2 Device registration on Cloud Industries procedure flow diagram.

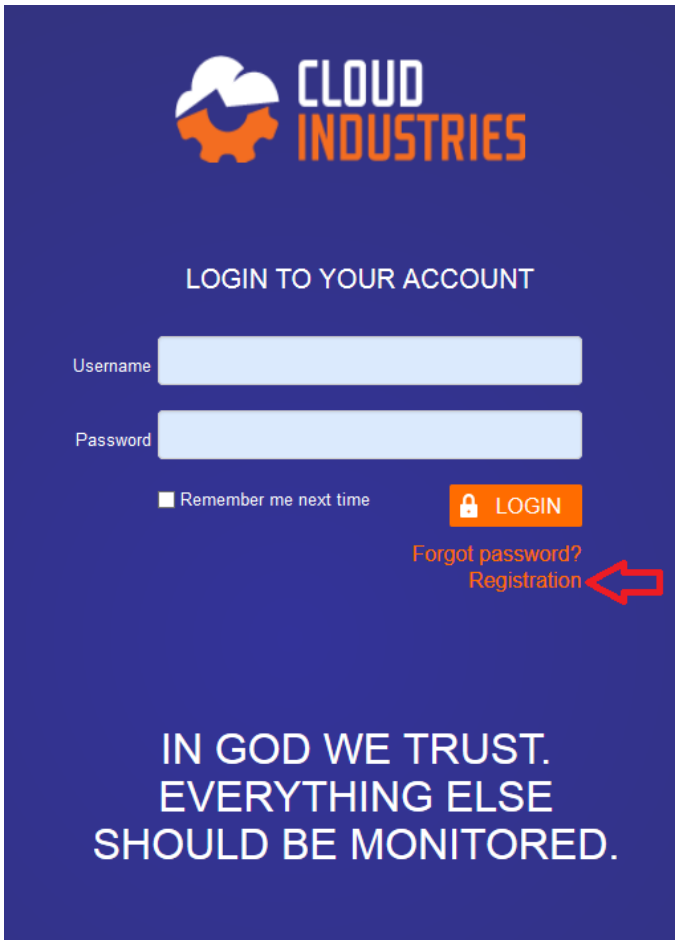
2. End-user login

2.1 Generating DUID and PIN code pair

An excel spreadsheet is supplied with programmed module to generate a PIN code. Each PIN code is unique for the serial number. You should supply each user with Serial number and it's PIN code. Chapter 2.2 contains instructions how user should perform registration on our system.

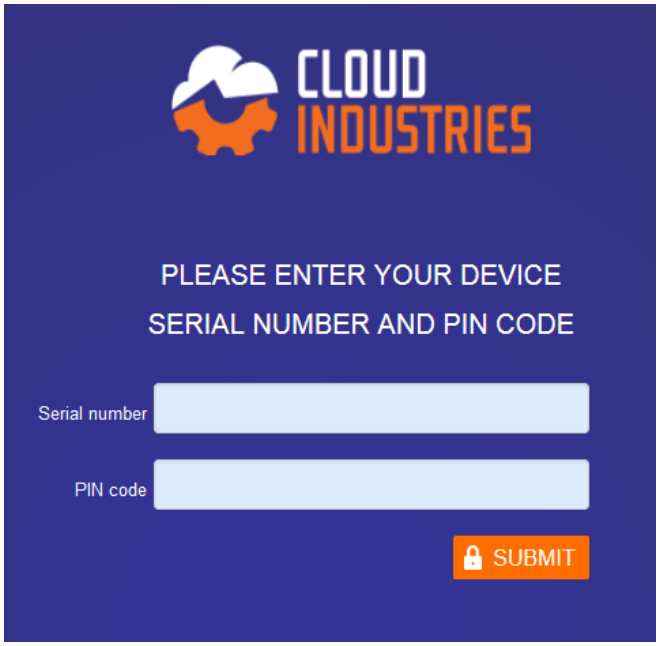
2.2 User registration

After user has received the device he should connect to https://inresol.cloudindustries.eu and click register link below login form (ref. Picture 2.1).



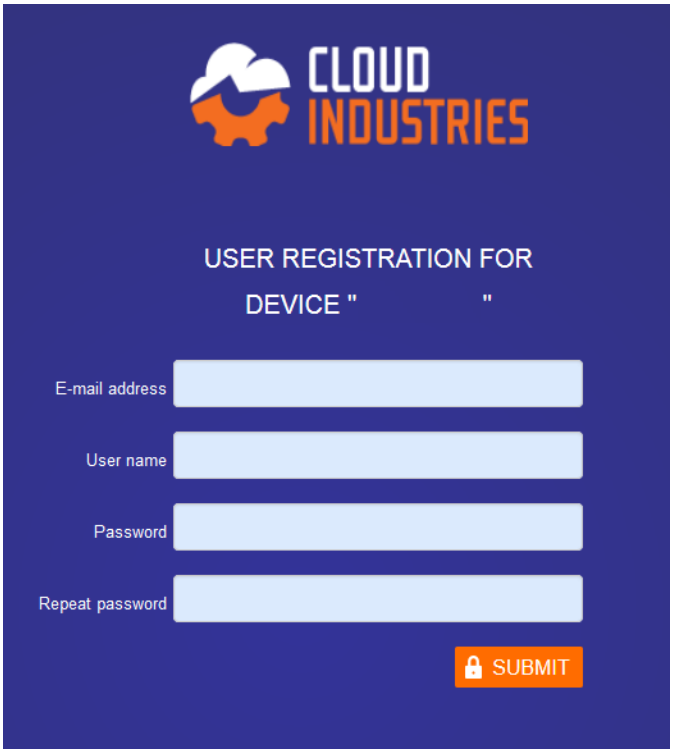
Pict. 2.1. Registration button in the user login form.

When registration link is clicked serial number and PIN code form is shown (ref. Picture 2.2). User should enter the device number and PIN code generated from the PIN code generation spreadsheet.



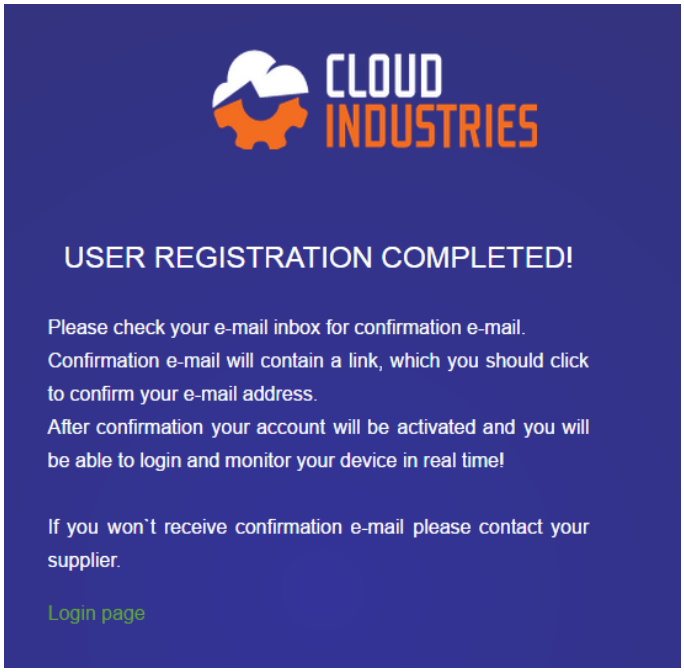
Pict. 2.2. Serial number and PIN code submition form.

If user has entered correct values and submitted them a user registration form will appear (Ref. Picture 2.3). User should enter his e-mail address, desired user name and password.

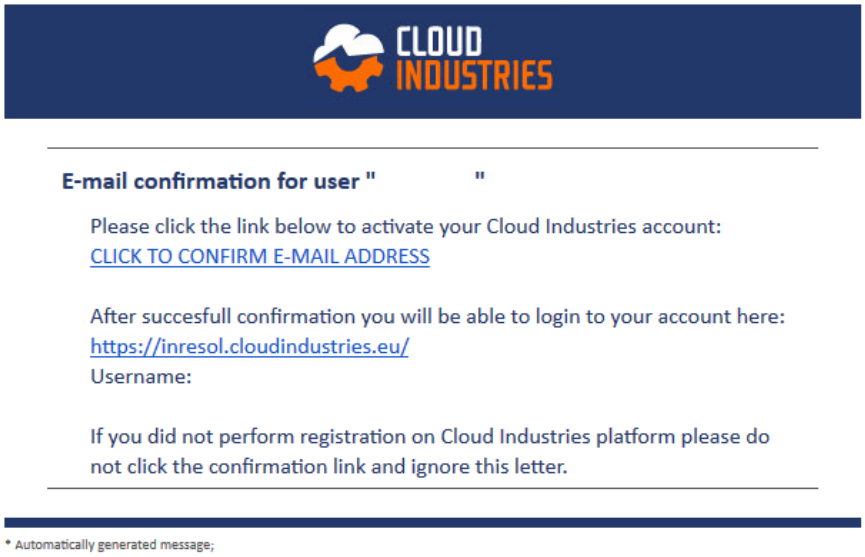


Pict. 2.3. User registration form.

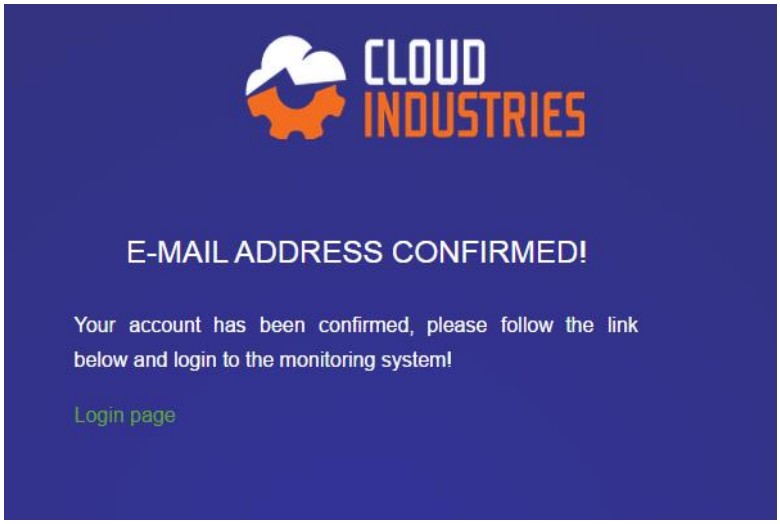
After successful submition of user credentials, a confirmation message will appear (Ref. Pict. 2.4) and an e-mail address confirmation request will be sent to user‘s e-mail (Ref. Pict. 2.5).



Pict. 2.4. User registration confirmation message.



Pict. 2.5. Confirmation e-mail request sent to user.



Pict. 2.6. User account activation message.

When the e-mail address is confirmed user account will be activated immediately and user should be able to login.

3. Cloud Industries API

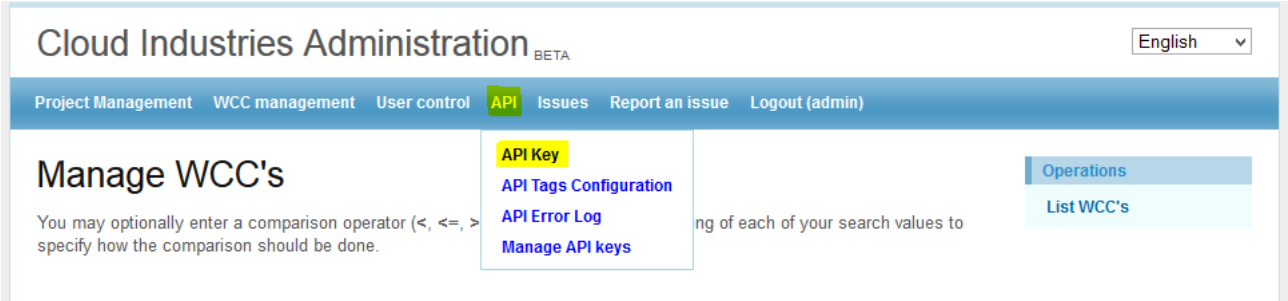
Cloud Industries server uses a RESTful API for data transmission. API is available through HTTP protocol and most of the functions is covered in the Cloud Industries connection library. Our API allows external devices to send data and events to the Cloud Industries system.

API key is used to authenticate HTTP requests. This key should be distributed among the devices and each device should use the same API key. Each device is identified by it‘s unique ID and authenticating using the API key. Chapter 3.1 describes how to find the key and unique device ID‘s inside the Cloud Industries administration tool.

3.1. Prerequisites

3.1.1. API Key

API Key is required when you want to make a HTTP request to the server API. API Key is available through Cloud Industries Administration tool. The key will be available after login, you can find it under the „**API**“ -> „**API Key**“ menu.



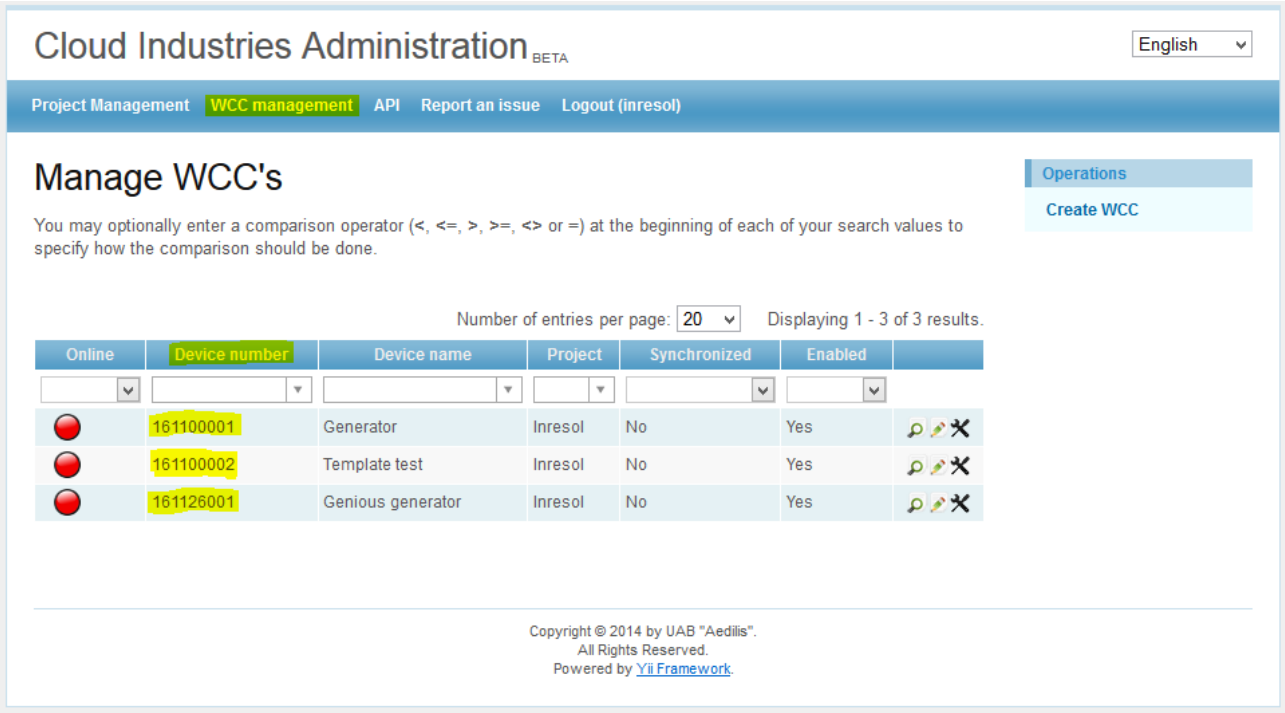
Pict. 3.1. API Key menu section

Pict. 3.2. API Key view

There are two types of keys, they differ only in length. Long API key is safer to use, but sometimes memory limited devices may be unable to handle such long strings for login headers so a shorter API key may be used.

3.1.2. Unique device ID (DUID)

Before starting the use of developer tools you should know the DUID of device you are going to work with. If the device is already registered on the system, you can check the DUID number in the administration tools. On the „**WCC Management**“ menu section. If you do not have or know the DUID you ca use DUID register request to create a new device in the system.



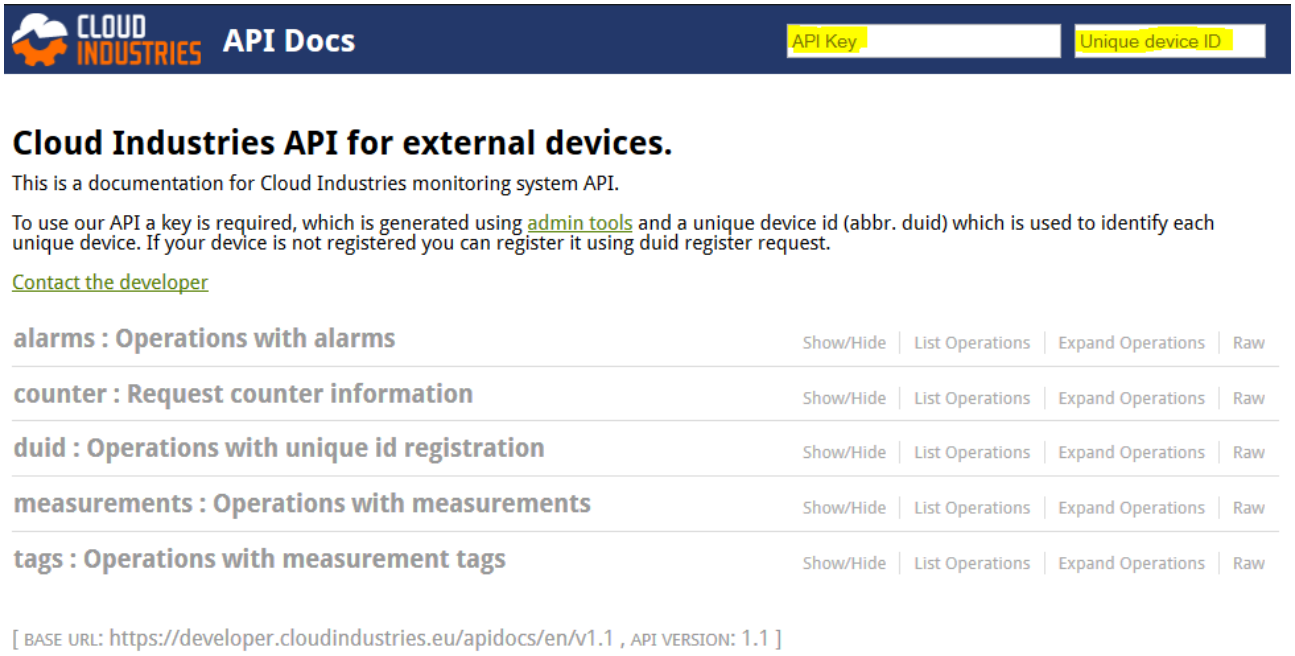
Pict 3.3. Device list with DUID identification numbers.

3.2. API Documentation and testing environment.

The Cloud Industries API Documentations and testing environment is available at https://developer.cloudindustries.eu. The documentation covers possible requests to our API and required information (request url, method, arguments) to execute the request.

In the picture below (ref. Picture) marked input fields are required for the development testing environment to work. You must supply your API Key (ref. Picture 3.2) and Device unique ID (ref. Picture 3.3). Then environment will allow the user to send API requests and analyse what results are available. This should come in handy if your device is unable to send the data (maybe a specific request) and you want analyse what the cause may be. After sending a request, response message will have a success message or error message if any occured.

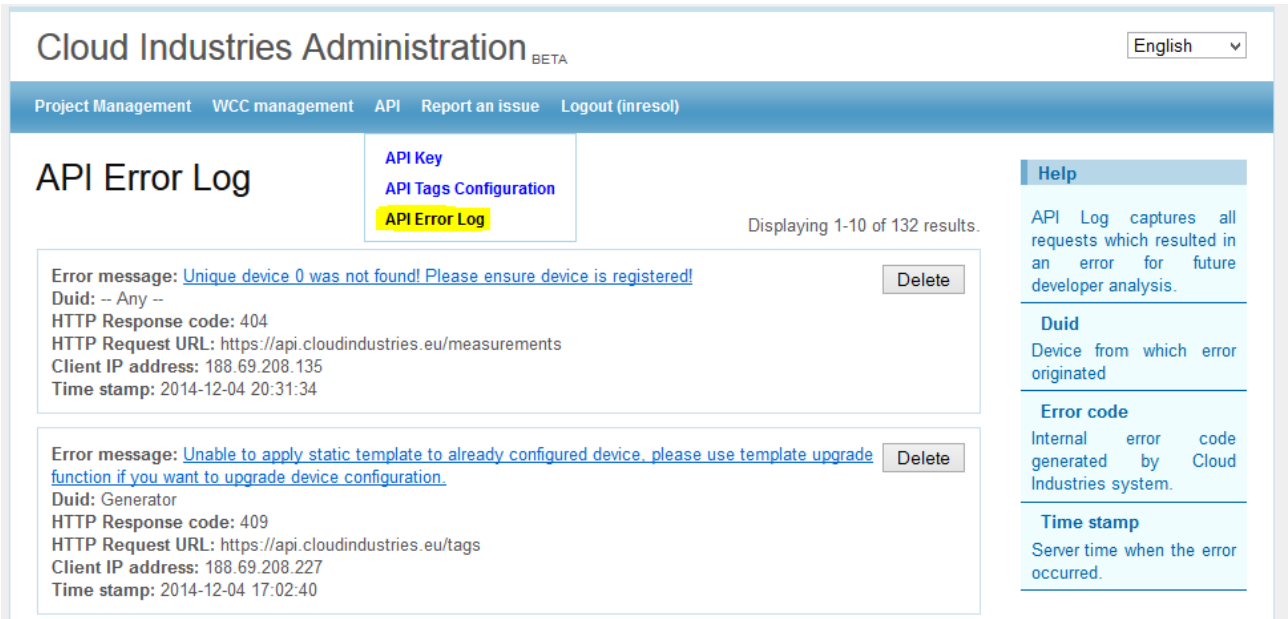
Feel free to take a look around. What possible request you can make and what is the possible results.



Pict. 3.4. API Documentation and testing environment UI.

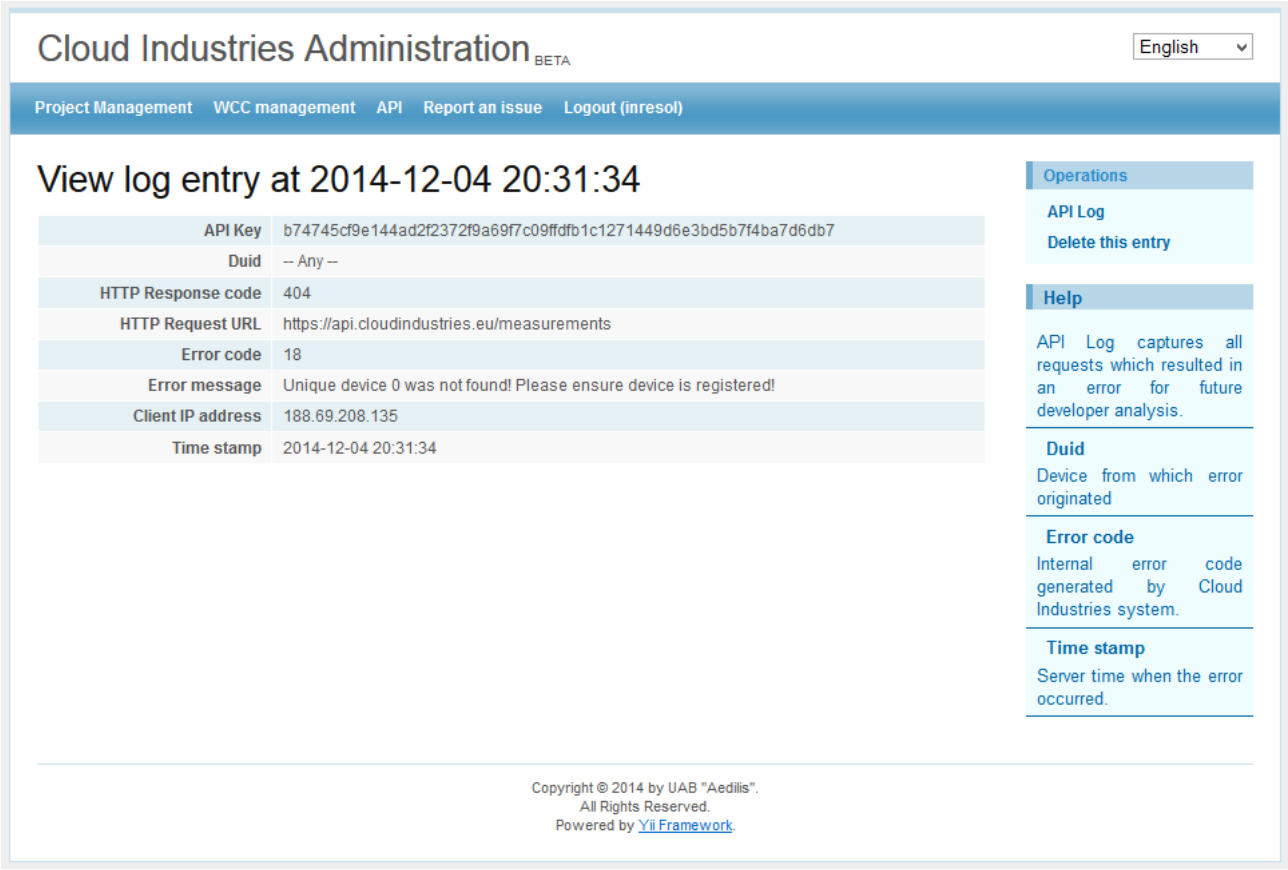
3.3. API Error logging

Since you are creating a system which will have remote device we have created an error logging tool which may help you discover problems when the device is already deployed.



Pict. 3.5. API Error log UI.

By clicking the link in the error message, a more detailed error information is available (ref Picture 3.6). The error displayed in the picture mentions that device sent it‘s DUID equal to zero. It is possible that device may not have a serial number or it does not send DUID number with the request to server.



Pict. 3.6. Detailed error information.